Rockchip HDMI-CEC Developer Guide

ID: RK-KF-YF-143

Release Version: V1.1.1

Release Date: 2025-04-21

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025-2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document mainly introduces the basic concepts related to CEC, the CEC-related software process based on Android 5.X and above, as well as Linux kernel 4.4/4.19, and common Debug methods.

Product Version

Chip Models	Kernel Version
RK322X/RK3328/RK3368/RK3399/RK3288	Linux kernel 4.4/4.19

Target Audience

This document (this guide) is mainly intended for:

Technical Support Engineers

Software Development Engineers

Revision Record

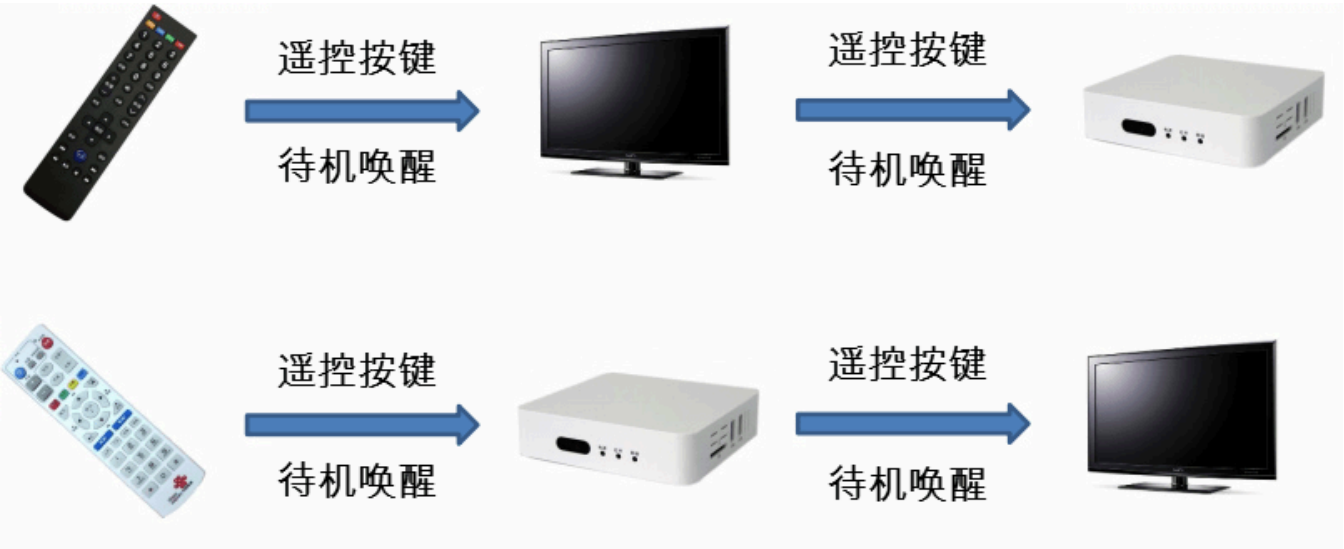
Date	Version	Author	Modification Description
2020-06-24	V1.0.0	Cao Jierui	Initial Release
2020-08-11	V1.1.0	Ruby Zhang	Added section 2.2 Linux User Layer Introduction
2025-04-21	V1.1.1	Karen Huang	Modified Document ID

Rockchip HDMI-CEC Developer Guide
1. Introduction to CEC
    1.1 Definition of CEC
    1.2 Introduction to the CEC Protocol
2. CEC Code Introduction
    2.1 Introduction to Android CEC Framework
        2.1.1 Overview of Android CEC Framework
            2.1.1.1 HdmiControlService
            2.1.1.2 HDMI-CEC HAL
        2.1.2 Relevant Code Paths
    2.2 Linux HDMI CEC Application Guide
        2.2.1 Installation of v4l-utils
        2.2.2 Related Commands
    2.3 Precautions
    2.4 Introduction to CEC Software Process
        2.4.1 CEC Initialization Process
            2.4.1.1 CEC Driver Registration Process Introduction
            2.4.1.2 CEC Initialization Setup Process Overview
        2.4.2 Introduction to CEC Message Sending Process
        2.4.3 Introduction to CEC Message Reception Process
        2.4.4 CEC Event Handling Process
    2.5 New CEC FEATURE Common Interface
3. Common Debug Methods
    3.1 DEBUG Script
        3.1.1 Log Content Introduction
        3.1.2 CEC DEBUG Node Description
        3.1.3 How to Enable CEC Kernel Log
    3.2 Troubleshooting
        3.2.1 Scenarios Where All CEC Functions Are Inoperative
        3.2.2 TV in Standby Mode, Box Not Entering Standby
        3.2.3 Box in Standby, TV Not Entering Standby
        3.2.4 Box Wakes Up, TV Not Woken Up
        3.2.5 TV Remote Control Cannot Control the Box's UI
4. Reference Documents

1. Introduction to CEC

1.1 Definition of CEC

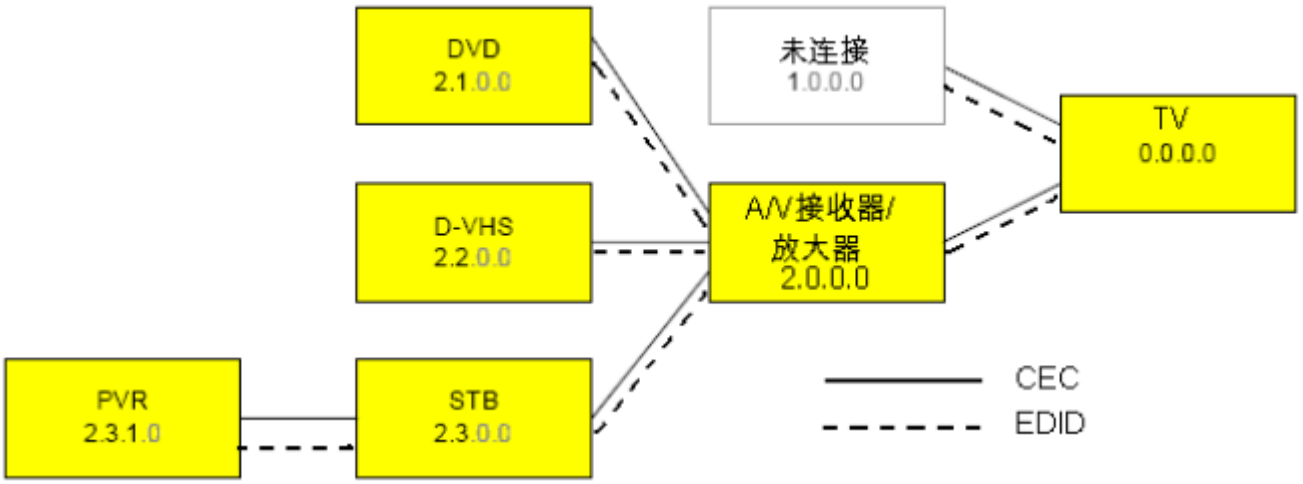
CEC stands for Consumer Electronics Control, a protocol that provides advanced control functions for all home audio-visual devices connected through HDMI cables within a user's environment, allowing users to control these connected devices with a single remote control.



As shown in the figure, it is possible to control both the TV and the box with just the TV remote control, or control both the TV and the box with just the box remote control.

1.2 Introduction to the CEC Protocol

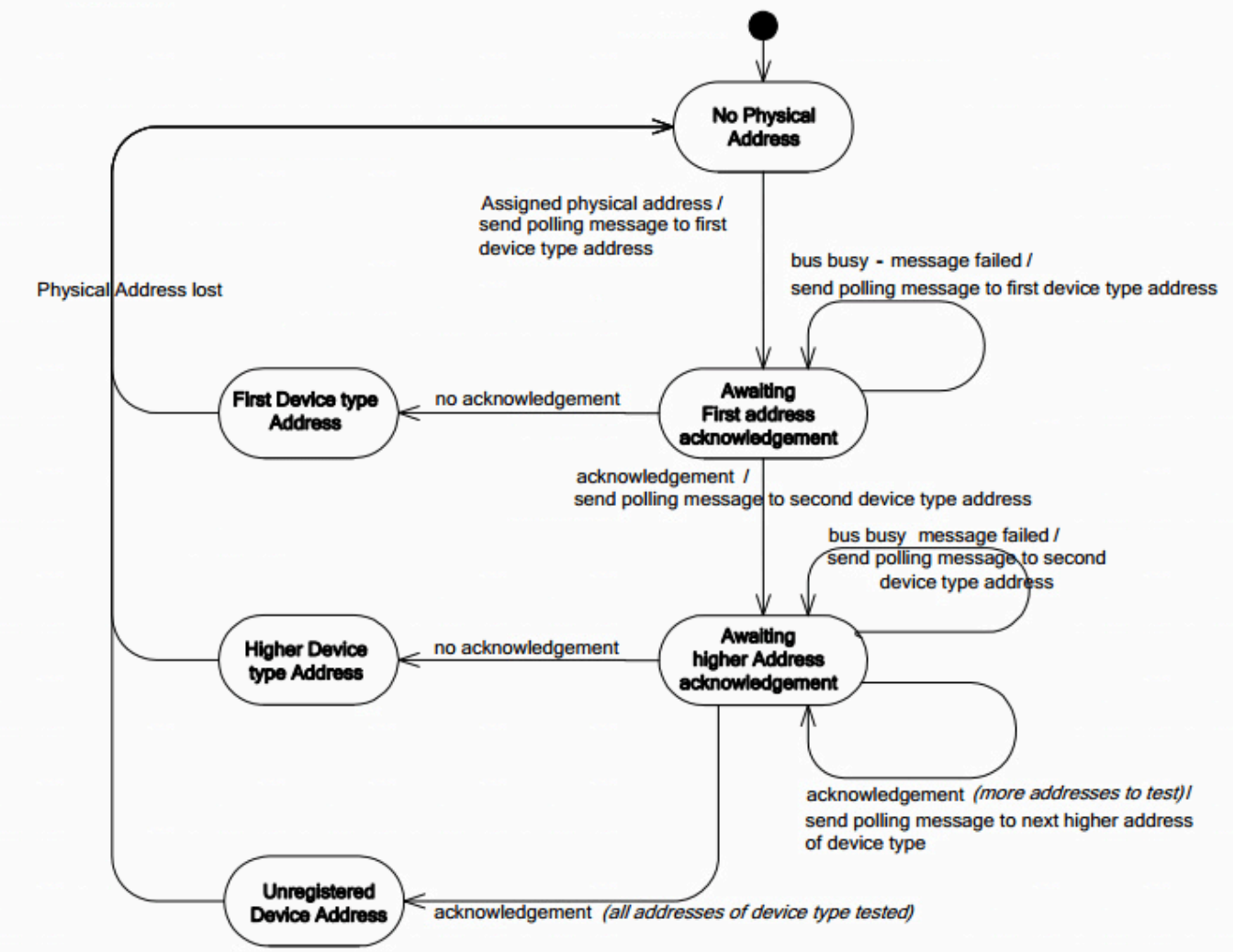
CEC assumes that all audio and video signal source products within a system are directly or indirectly connected to a "root" display device, forming a top-down tree structure connected by HDMI cables, with the display device acting as the "root," signal switching devices as "branches," and various signal source products as "leaf" nodes.
To enable CEC to address and control devices with specific physical addresses, all devices within the system must possess physical addresses. Software assigns physical addresses to all devices in the CEC network through EDID. Each device has and can only have one physical address.



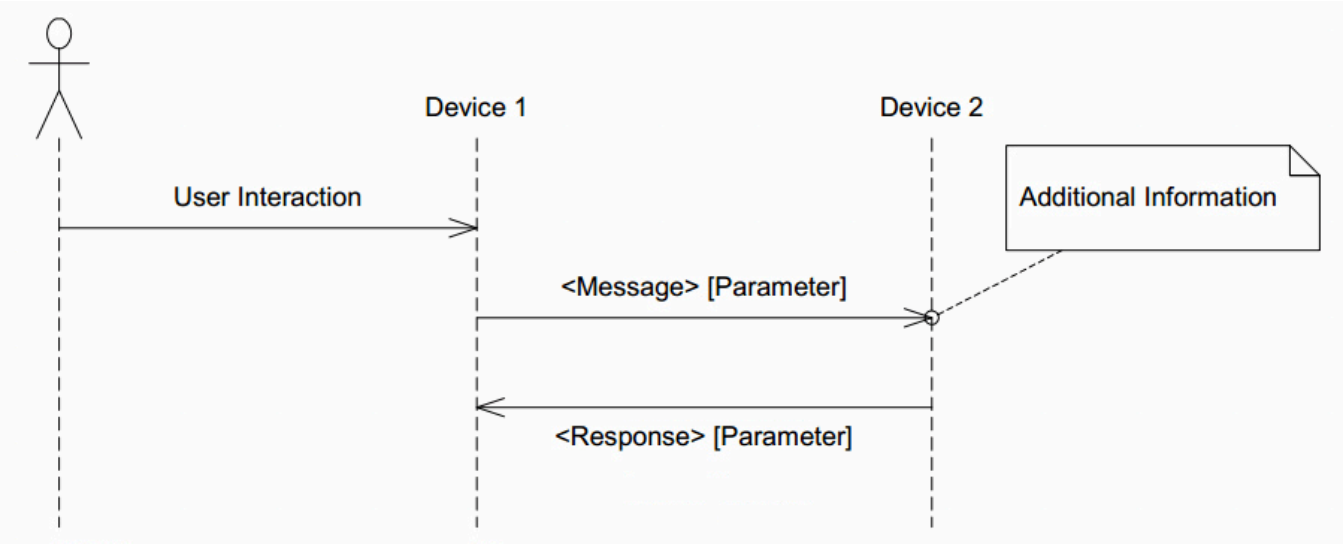
Every device connected to the CEC control bus must bind to a logical address, defining the type of the device. Each logical address can only be bound to a single device (except for 15). Most devices bind to only one logical address, while a few devices can bind to a maximum of two logical addresses.

Address	Device
0	TV
1	Recording Device 1
2	Recording Device 2
3	Tuner 1
4	Playback Device 1
5	Audio System
6	Tuner 2
7	Tuner 3
8	Playback Device 2
9	Recording Device 3
10	Tuner 4
11	Playback Device 3
12	Reserved
13	Reserved
14	Specific Use
15	Unregistered (as Initiator address) Broadcast (as Destination address)

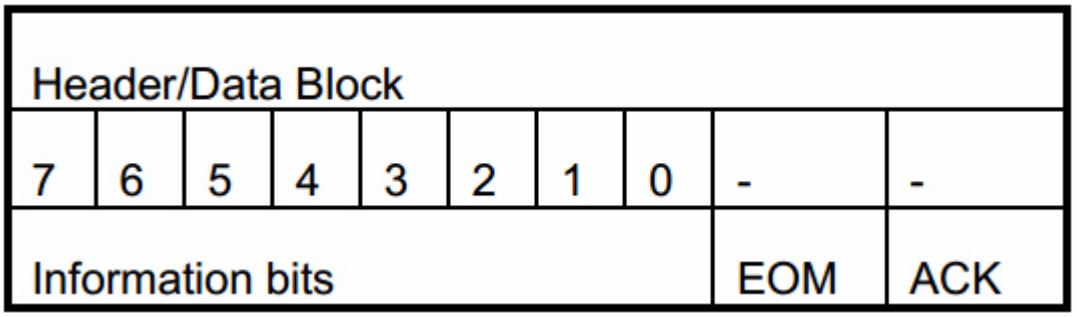
The process of binding logical addresses is illustrated in the figure below, where the address is confirmed to be unoccupied by sending a POLL MSG with the same SRC and DST addresses.



Communication between CEC devices is achieved by sending CEC MSGs.



CEC MSGs consist of one or more Blocks, with the Block format as follows, composed of an 8-bit Header/Data, a 1-bit EOM, and a 1-bit ACK bit. The EOM indicates whether there is more data to follow; when set to 1, it signifies that the CEC MSG has ended and there is no more data to follow. The ACK is an acknowledgment bit; the sender sets it to 1, and if the receiver successfully receives the MSG, it sets it to 0, indicating that the data has been received.



The Header Block is the first Block of the CEC MSG, with the high 4 bits of the Information bits representing the Logical address of the sender and the low 4 bits representing the Logical address of the receiver. The Information bits of the Data Block are either the OPCODE or parameters.

2. CEC Code Introduction

2.1 Introduction to Android CEC Framework

2.1.1 Overview of Android CEC Framework

Many manufacturers have adopted the HDMI-CEC standard, enabling devices from different manufacturers to work together. However, due to the varying implementations of HDMI-CEC by each manufacturer and the differences in supported features, they are not fully compatible. Consumers cannot simply assume that two products claiming CEC support are compatible with each other.
With the introduction of Android TIF (TV Input Framework), HDMI-CEC allows interconnected devices to communicate and minimizes compatibility issues. Android has created a system service, HdmiControlService, to address this pain point.
By providing HdmiControlService as part of the Android ecosystem, Android aims to achieve the following goals:
To offer a standard implementation of HDMI-CEC for all manufacturers, which will reduce incompatibilities between devices. Previously, manufacturers had to develop their own HDMI-CEC or use third-party solutions. HdmiControlService is an Android service that has been well-tested for the extensive HDMI-CEC devices currently used in the market. Android has conducted thorough research on compatibility issues and has collected many useful suggestions from experienced industry partners. This CEC service strikes a good balance between standard compliance and real-world compatibility, so it can be used with products that users already own.

2.1.1.1 HdmiControlService

HdmiControlService works alongside other parts of the system (such as TIF, Audio services, power management services, etc.) to implement the CEC standard. The diagram describes how to transition from a previously custom CEC controller to the current simpler HDMI-CEC hardware abstraction layer. It also provides a detailed description of the implementation of the HDMI control service.
Below are the key components of the Android HDMI-CEC implementation:

- A management class HdmiControlManager provides APIs to applications with permissions. System

services such as TV Input Manager services and Audio services can directly use this class.

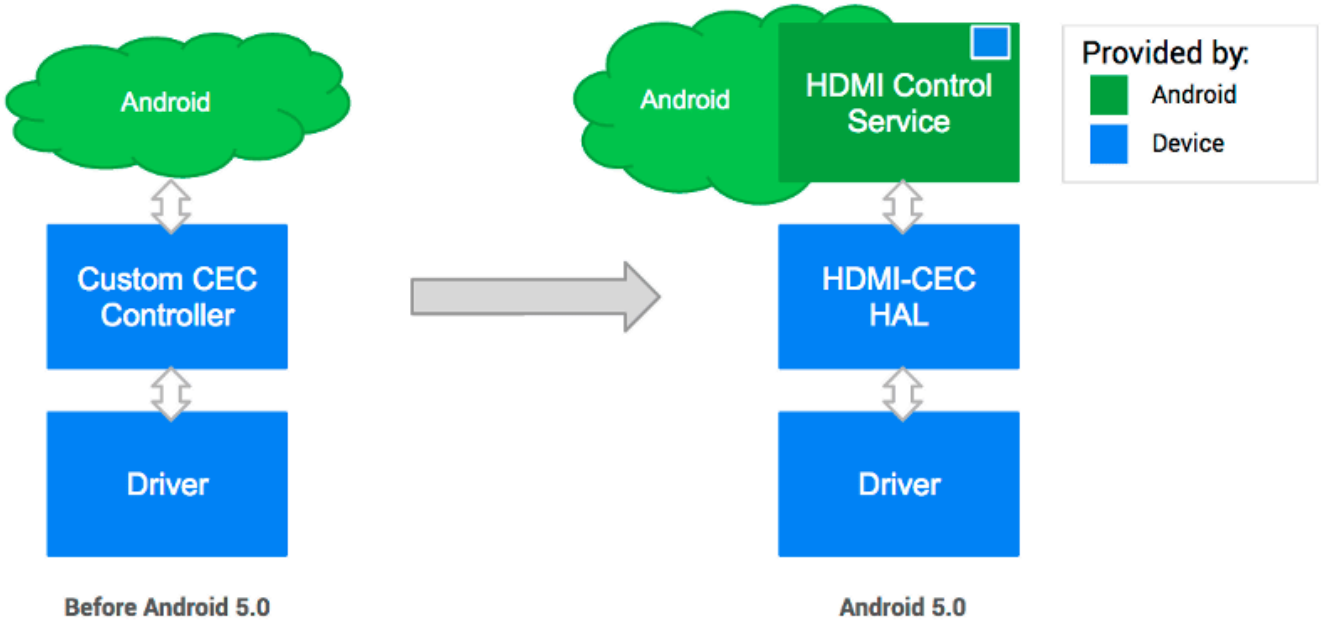
- Code location:

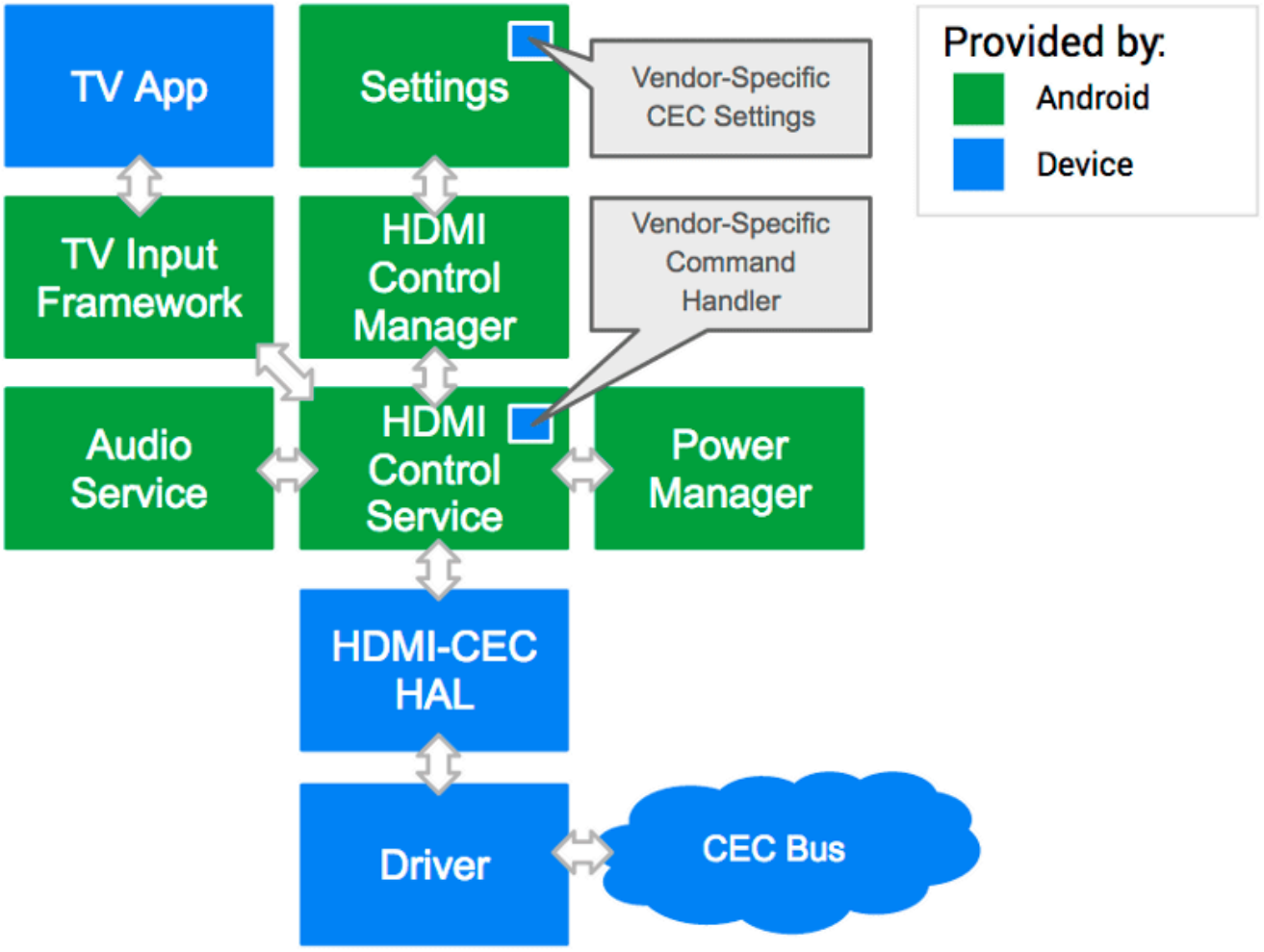
```
frameworks/base/core/java/android/hardware/hdmi
```

- This service is designed to be capable of supporting multiple types of logical devices.
- Code location:

```
frameworks/base/services/core/java/com/android/server/hdmi
```

- HDMI-CEC operates through the hardware abstraction layer, which simplifies the differences in protocols and signaling mechanisms between devices. Manufacturers can leverage the existing definitions of the hardware abstraction layer to implement their own hardware abstraction layer.





2.1.1.2 HDMI-CEC HAL

To enable the CEC service to function effectively, it is imperative to implement the HDMI-CEC HAL according to the definitions provided by Android. The HDMI-CEC HAL abstracts the hardware-level differences and offers raw operations (allocation/reading/writing, etc.) to the upper layer through APIs.
Manufacturers must support the following API calls:
TX/RX/Events:

- send_message
- register_event_callback

Information:

- get_physical_address
- get_version
- get_vendor_id
- get_port_info

Logical Address:

- add_logical_address
- clear_logical_address

Status:

- is_connected
- set_option
- set_audio_return_channel

Here is an excerpt of the HDMI-CEC HAL API definition:

```
#ifndef ANDROID_INCLUDE_HARDWARE_HDMI_CEC_H
#define ANDROID_INCLUDE_HARDWARE_HDMI_CEC_H

...

/*
 * HDMI-CEC HAL interface definition.
 */
typedef struct hdmi_cec_device {
  /**
   * Common methods of the HDMI-CEC device.  This *must* be the first member of
   * hdmi_cec_device as users of this structure will cast a hw_device_t to hdmi_cec_device
   * pointer in contexts where it's known the hw_device_t references a hdmi_cec_device.
   */
  struct hw_device_t common;

  /*
   * (*add_logical_address) passes the logical address that will be used
   * in this system.
   *
   * HAL may use it to configure the hardware so that the CEC commands addressed
   * the given logical address can be filtered in. This method can be called
   * as many times as necessary in order to support multiple logical devices.
   * addr should be in the range of valid logical addresses for the call
   * to succeed.
   *
   * Returns 0 on success or -errno on error.
   */
  int (*add_logical_address)(const struct hdmi_cec_device* dev, cec_logical_address_t addr);

  /*
   * (*clear_logical_address) tells HAL to reset all the logical addresses.
   *
   * It is used when the system doesn't need to process CEC command any more,
   * hence to tell HAL to stop receiving commands from the CEC bus, and change
   * the state back to the beginning.
   */
  void (*clear_logical_address)(const struct hdmi_cec_device* dev);

  /*
   * (*get_physical_address) returns the CEC physical address. The
   * address is written to addr.
   *
   * The physical address depends on the topology of the network formed
```

```
     * by connected HDMI devices. It is therefore likely to change if the cable
     * is plugged off and on again. It is advised to call get_physical_address
     * to get the updated address when hot plug event takes place.
     *
     * Returns 0 on success or -errno on error.
     */
    int (*get_physical_address)(const struct hdmi_cec_device* dev, uint16_t* addr);

    /*
     * (*send_message) transmits HDMI-CEC message to other HDMI device.
     *
     * The method should be designed to return in a certain amount of time not
     * hanging forever, which can happen if CEC signal line is pulled low for
     * some reason. HAL implementation should take the situation into account
     * so as not to wait forever for the message to get sent out.
     *
     * It should try retransmission at least once as specified in the standard.
     *
     * Returns error code. See HDMI_RESULT_SUCCESS, HDMI_RESULT_NACK, and
     * HDMI_RESULT_BUSY.
     */
    int (*send_message)(const struct hdmi_cec_device* dev, const cec_message_t*);

    /*
     * (*register_event_callback) registers a callback that HDMI-CEC HAL
     * can later use for incoming CEC messages or internal HDMI events.
     * When calling from C++, use the argument arg to pass the calling object.
     * It will be passed back when the callback is invoked so that the context
     * can be retrieved.
     */
    void (*register_event_callback)(const struct hdmi_cec_device* dev,
            event_callback_t callback, void* arg);

    /*
     * (*get_version) returns the CEC version supported by underlying hardware.
     */
    void (*get_version)(const struct hdmi_cec_device* dev, int* version);

    /*
     * (*get_vendor_id) returns the identifier of the vendor. It is
     * the 24-bit unique company ID obtained from the IEEE Registration
     * Authority Committee (RAC).
     */
    void (*get_vendor_id)(const struct hdmi_cec_device* dev, uint32_t* vendor_id);

    /*
     * (*get_port_info) returns the hdmi port information of underlying hardware.
     * info is the list of HDMI port information, and 'total' is the number of
     * HDMI ports in the system.
     */
    void (*get_port_info)(const struct hdmi_cec_device* dev,
            struct hdmi_port_info* list[], int* total);
```

```
    /*
     * (*set_option) passes flags controlling the way HDMI-CEC service works down
     * to HAL implementation. Those flags will be used in case the feature needs
     * update in HAL itself, firmware or microcontroller.
     */
    void (*set_option)(const struct hdmi_cec_device* dev, int flag, int value);

    /*
     * (*set_audio_return_channel) configures ARC circuit in the hardware logic
     * to start or stop the feature. Flag can be either 1 to start the feature
     * or 0 to stop it.
     *
     * Returns 0 on success or -errno on error.
     */
    void (*set_audio_return_channel)(const struct hdmi_cec_device* dev, int flag);

    /*
     * (*is_connected) returns the connection status of the specified port.
     * Returns HDMI_CONNECTED if a device is connected, otherwise HDMI_NOT_CONNECTED.
     * The HAL should watch for +5V power signal to determine the status.
     */
    int (*is_connected)(const struct hdmi_cec_device* dev, int port);

    /* Reserved for future use to maximum 16 functions. Must be NULL. */
    void* reserved[16 - 11];
} hdmi_cec_device_t;

#endif /* ANDROID_INCLUDE_HARDWARE_HDMI_CEC_H */
```

With the help of APIs, the CEC service can utilize hardware resources to send/receive HDMI-CEC commands, configure necessary settings, and (optionally) communicate with the microprocessor on the underlying platform (which will be responsible for CEC control when the Android system is in standby mode).

2.1.2 Relevant Code Paths

The code paths related to CEC functionality and their descriptions are as follows:

Path	Description
kernel/drivers/media/cec/cec-adap.c	The core part of the CEC driver, responsible for binding CEC Physical address and Logical address, managing CEC EVENT and MSG transmission, etc.
kernel/drivers/media/cec/cec-api.c	Provides IOCTL for USER SPACE to call.
kernel/drivers/media/cec/cec-core.c	CEC device registration.
kernel/drivers/media/cec/cec-notifier.c	Notifies the change of CEC driver Physical address.
kernel/drivers/media/cec/cec-edid.c	CEC EDID auxiliary function. Mainly for obtaining Physical address from EDID.
kernel/drivers/gpu/drm/bridge/synopsys/dw-hdmi-cec.c	DW-HDMI CEC driver. Mainly implements operations on DW-HDMI CEC registers.

Android code paths are as follows:

Path	Description
hardware/rockchip/hdmicec/hdmi_cec.cpp	Implements the standard CEC HAL interface.
hardware/rockchip/hdmicec/hdmicec_event.cpp	Mainly achieves listening to kernel CEC MSG and EVENT through a thread, and reporting upwards.
frameworks/base/services/core/java/com/android/server/hdmi/HdmiControlService.java	Provides a service for sending and processing HDMI control messages and CEC control commands.
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecController.java	Manages CEC commands and operations. Converts user commands into CEC commands and sends them to HAL. After receiving CEC messages, it parses and dispatches them to specific modules.
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecLocalDevice.java	Models a logical CEC device in the system, handling initialization and calling specific device processing interfaces after receiving CEC MSGs targeted at specific devices.
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java	Abstracts a PLAYBACK device, providing related interfaces.

2.2 Linux HDMI CEC Application Guide

Linux allows the use of the cec-ctl tool provided by v4l-utils to control CEC devices through the command line.

2.2.1 Installation of v4l-utils

- On Debian, users can install v4l-utils by executing the following command:

```
sudo apt-get install v4l-utils
```

- On Buildroot, users can install v4l-utils by configuring the following build options:

```
BR2_PACKAGE_LIBV4L=y
BR2_PACKAGE_LIBV4L_UTILS=y
```

2.2.2 Related Commands

- **Playback Command**:

```
[root@rk3288:/]# cec-ctl --playback -o Rockchip -V 0xaabbcc -M -T
```

Partial Output Log:

```
CEC_ADAP_G_CAPS returned 0 (Success)
CEC_ADAP_G_PHYS_ADDR returned 0 (Success)
CEC_ADAP_S_LOG_ADDRS returned 0 (Success)
CEC_ADAP_S_LOG_ADDRS returned 0 (Success)
CEC_ADAP_G_LOG_ADDRS returned 0 (Success)
Driver Info:
Driver Name : dwhdmi-rockchip
Adapter Name : dw_hdmi
Capabilities : 0x0000000e
Logical Addresses
Transmit
Passthrough
Driver version : 4.4.167
Available Logical Addresses: 4
Physical Address : 1.0.0.0
Logical Address Mask : 0x0010
CEC Version : 2.0
Vendor ID : 0xaabbcc
Logical Address : 4 (Playback Device 1)
Primary Device Type : Playback
Logical Address Type : Playback
All Device Types : Playback
RC TV Profile : None
Device Features : None
Monitor All mode is not supported, falling back to regular monitoring
CEC_S_MODE returned 0 (Success)
CEC_DQEVENT returned 0 (Success)
```

CEC initialization will involve basic information exchange, such as vendor id, osd name, CEC version:

```
Received from TV to Playback Device 1 (0 to 4): CEC_MSG_GIVE_DEVICE_VENDOR_ID (0x8c)
CEC_RECEIVE returned 0 (Success)
Transmitted by Playback Device 1 to all (4 to 15): CEC_MSG_DEVICE_VENDOR_ID (0x87):
vendor-id: 11189196 (0x00aabbcc)
CEC_RECEIVE returned 0 (Success)

Received from TV to Playback Device 1 (0 to 4): CEC_MSG_GIVE_OSD_NAME (0x46)
CEC_RECEIVE returned 0 (Success)
Transmitted by Playback Device 1 to TV (4 to 0): CEC_MSG_SET_OSD_NAME (0x47):
name: Rockchip
CEC_RECEIVE returned 0 (Success)
```

When the TV is in standby mode, it will send standby information to the chip:

```
Received from TV to all (0 to 15): CEC_MSG_STANDBY (0x36)
CEC_RECEIVE returned 0 (Success)
```

When the TV switches the display source, it will send the relevant information to the chip:

```
Received from TV to all (0 to 15): CEC_MSG_SET_STREAM_PATH (0x86):
phys-addr: 2.0.0.0
CEC_RECEIVE returned 0 (Success)
Received from TV to all (0 to 15): CEC_MSG_ROUTING_CHANGE (0x80):
orig-phys-addr: 0.0.0.0
new-phys-addr: 2.0.0.0
CEC_RECEIVE returned 0 (Success)
```

- One-touch-play Command:

```
[root@rk3288:/]# cec-ctl --help-one-touch-play
One Touch Play Feature:
--active-source=phys-addr=<val> Send ACTIVE_SOURCE message (0x82)
--image-view-on Send IMAGE_VIEW_ON message (0x04)
--text-view-on Send TEXT_VIEW_ON message (0x0d)
```

Wake up TV:

```
[root@rk3288:/]# cec-ctl --image-view-on -to 0
```

- Standby Command:

```
[root@rk3288:/]# cec-ctl --standby --to 0
```

For more commands in the CEC user layer, you can obtain them through `cec-ctl --help`.

2.3 Precautions

- Currently, it is not supported to wake up the chip from standby through the TV's standby wake-up operation.

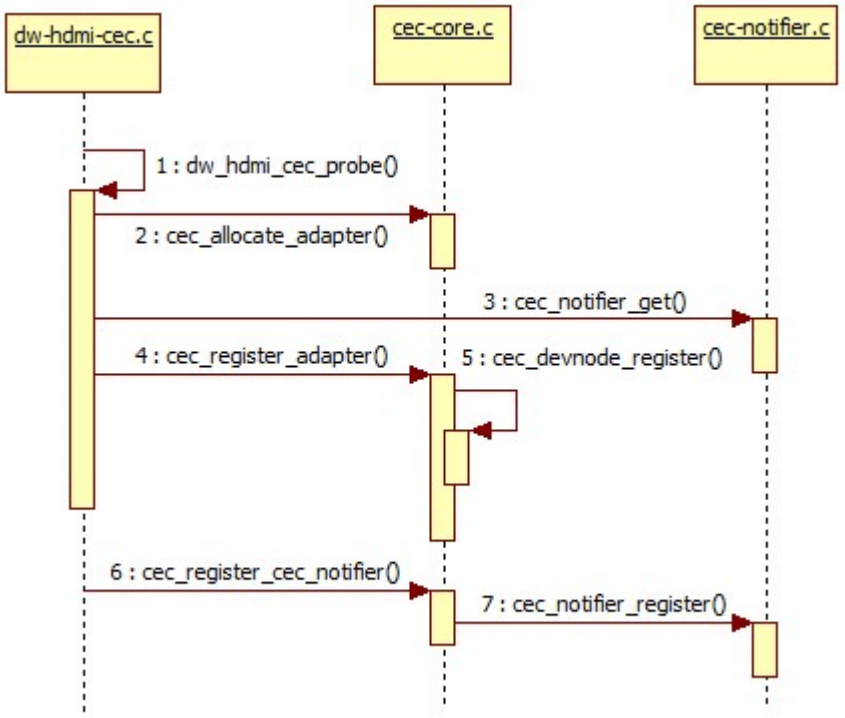
Standby can refer to the Playback Command section above. Upon receiving CEC_MSG_STANDBY, executing the following shell command can achieve chip-side synchronous standby:
`echo mem > /sys/power/state`
Wake-up involves TrustZone and operations such as cec-clk, hdmi phy during standby, and currently, support is not yet fully implemented.

- Not all HDMI devices support CEC functionality, please confirm whether the TV end or HDMI display device supports CEC and whether they support specific CEC commands.

2.4 Introduction to CEC Software Process

2.4.1 CEC Initialization Process

2.4.1.1 CEC Driver Registration Process Introduction



The CEC driver registration process is shown in the figure above.

- dw_hdmi_cec_probe:

Completes the initialization of the CEC driver, linking the system-abstracted CEC device with the actual CEC hardware. It also includes the initialization configuration of CEC-related registers and the registration of interrupts for CEC MSG transmission and reception (cec->irq), etc.

- cec_allocate_adapter:

(1) Completes the creation and adaptation of the adapter.

The adapter structure is as follows:

```
struct cec_adapter {
    struct module *owner;
    char name[32];
    struct cec_devnode devnode;
    struct mutex lock;
    struct rc_dev *rc;

    struct list_head transmit_queue;
    unsigned int transmit_queue_sz;
    struct list_head wait_queue;
    struct cec_data *transmitting;
    bool transmit_in_progress;

    struct task_struct *kthread_config;
    struct completion config_completion;

    struct task_struct *kthread;
    wait_queue_head_t kthread_waitq;
    wait_queue_head_t waitq;

    const struct cec_adap_ops *ops;
    void *priv;
    u32 capabilities;
    u8 available_log_addrs;

    u16 phys_addr;
    bool needs_hpd;
    bool is_configuring;
    bool is_configured;
    bool cec_pin_is_high;
    u8 last_initiator;
    u32 monitor_all_cnt;
    u32 monitor_pin_cnt;
    u32 follower_cnt;
    struct cec_fh *cec_follower;
    struct cec_fh *cec_initiator;
    bool passthrough;
    struct cec_log_addrs log_addrs;

    u32 tx_timeouts;

#ifdef CONFIG_CEC_NOTIFIER
    struct cec_notifier *notifier;
#endif
#ifdef CONFIG_CEC_PIN
    struct cec_pin *pin;
#endif
```

```
    struct dentry *cec_dir;
    struct dentry *status_file;
    struct dentry *error_inj_file;

    u16 phys_addrs[15];
    u32 sequence;

    char device_name[32];
    char input_phys[32];
    char input_drv[32];
};
```

Explanation of some of the more important structure members is given in the table below:

Name	Description
transmit_queue	Queue for CEC MSGs waiting to be sent.
transmit_queue_sz	Length of the queue for CEC MSGs waiting to be sent.
wait_queue	Messages waiting for a response to a sent CEC MSG are stored in this queue; this feature is not used in the current version.
transmitting	The CEC MSG currently being sent
kthread_config	Thread running during CEC initialization.
config_completion	Semaphore waiting for CEC initialization to complete.
kthread	Descriptor of the thread managing the CEC MSG send queue, cec_thread_func.
kthread_waitq	Wait queue for cec_thread_func.
ops	Adapter callbacks, see kernel/drivers/gpu/drm/bridge/synopsys/dw-hdmi-cec.c
capabilities	Functions set for the adapter.
available_log_addrs	Maximum number of Logical addresses that can be obtained.
phys_addr	Current Physical address.
is_configuring	Currently undergoing initialization configuration.
is_configured	Initialization configuration is complete.
follower_cnt	Number of followers, currently set to 1.
cec_follower	Current cec_follower, currently fh.
cec_initiator	Current cec_initiator, currently fh.
passthrough	Whether the current cec_follower is in passthrough mode.
log_addrs	Currently bound Logical addresses, one or more.
tx_timeouts	Number of times CEC MSG transmission has timed out, which rarely occurs.
notifier	CEC notifier
phys_addrs	Physical address, for situations with multiple Logical addresses.
sequence	Sequence number for adapter sending CEC MSGs, used to trace waiting for reply MSGs.

(2) Runs the thread cec_thread_func that manages the CEC MSG transmission queue.

(3) Configures the capabilities of the cec adapter, detailed explanation is given in the table below:

capabilities	Description
CEC_CAP_PHYS_ADDR	The user layer must set the Physical address.
CEC_CAP_LOG_ADDRS	The user layer must set the Logical address.
CEC_CAP_TRANSMIT	Allows the user layer to pass CEC MSGs.
CEC_CAP_PASSTHROUGH	The CEC driver does not process received CEC MSGs and directly reports them to user space.
CEC_CAP_RC	Supports remote control driving CEC.
CEC_CAP_MONITOR_ALL	The CEC driver receives all CEC MSGs, including those not sent to itself, usually used for DEBUG purposes.
CEC_CAP_NEEDS_HPD	CEC is enabled only when the HDMI HPD pin is high.
CEC_CAP_MONITOR_PIN	The CEC driver monitors changes in the CEC pin.

- cec_register_adapter:

(1) Completes the registration of the CEC device node.

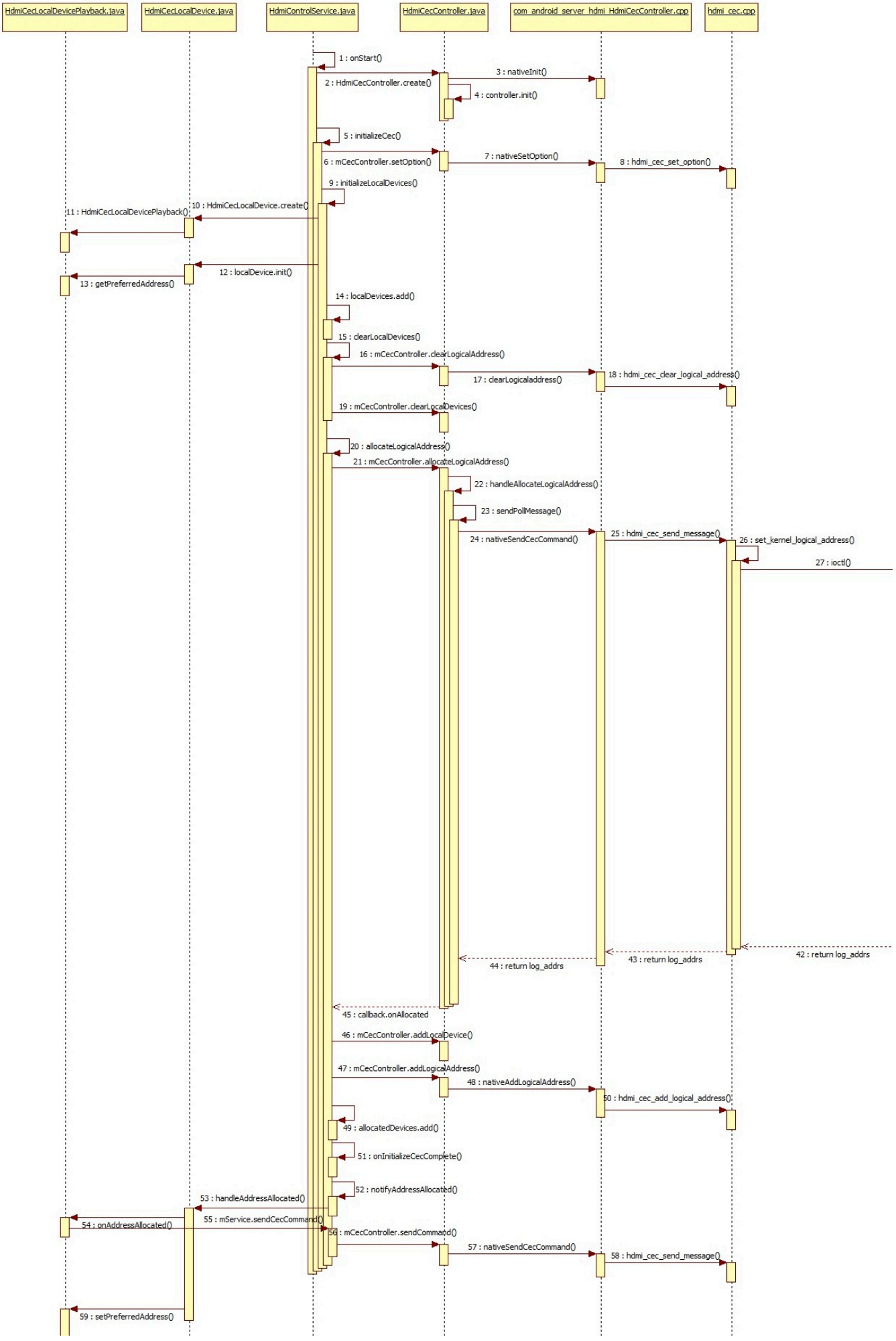
(2) Completes the registration of the CEC DEBUG node.

- cec_register_cec_notifier:

Registers the cec notifier and binds it with the adapter.

2.4.1.2 CEC Initialization Setup Process Overview

For CEC functionality to operate correctly, proper initialization configuration is required. This involves enabling the CEC function-related switches; binding and saving the Logical address; and completing the CEC driver configuration. The processes related to FRAMEWORK, JNI, and HAL are shown in the figure below:


HdmiCecLocalDevicePlayback.java
HdmiCecLocalDevice.java
HdmiControlService.java
HdmiCecController.java
com_android_server_hdmi_HdmiCecController.cpp
hdmi_cec.cpp
1 : onStart()
2 : HdmiCecController.create()
3 : nativeInit()
4 : controller.init()
5 : initializeCec()
6 : mCecController.setOption()
7 : nativeSetOption()
8 : hdmi_cec_set_option()
9 : initializeLocalDevices()
10 : HdmiCecLocalDevice.create()
11 : HdmiCecLocalDevicePlayback()
12 : localDevice.init()
13 : getPreferredAddress()
14 : localDevices.add()
15 : clearLocalDevices()
16 : mCecController.clearLogicalAddress()
17 : clearLogicaladdress()
18 : hdmi_cec_clear_logical_address()
19 : mCecController.clearLocalDevices()
20 : allocateLogicalAddress()
21 : mCecController.allocateLogicalAddress()
22 : handleAllocateLogicalAddress()
23 : sendPollMessage()
24 : nativeSendCecCommand()
25 : hdmi_cec_send_message()
26 : set_kernel_logical_address()
27 : ioctl()
42 : return log_addrs
43 : return log_addrs
44 : return log_addrs
45 : callback.onAllocated
46 : mCecController.addLocalDevice()
47 : mCecController.addLogicalAddress()
48 : nativeAddLogicalAddress()
50 : hdmi_cec_add_logical_address()
49 : allocatedDevices.add()
51 : onInitializeCecComplete()
52 : notifyAddressAllocated()
53 : handleAddressAllocated()
54 : onAddressAllocated()
55 : mService.sendCecCommand()
56 : mCecController.sendCommand()
57 : nativeSendCecCommand()
58 : hdmi_cec_send_message()
59 : setPreferredAddress()

The more important steps include:

- initializeCec:

initializeCec is the most critical step in the initialization process. It is not only executed during the boot initialization but also when the system wakes up from standby or when CEC functionality is enabled in the settings menu. It mainly completes the initialization of the local device (currently, Android only supports TV and PLAYBACK devices, with RK solutions primarily being PLAYBACK devices), the binding of the Logical address, and the enabling of the CEC switch.

- initializeLocalDevices:

(1) A CEC local device will be created and initialized, specifically creating HdmiCecLocalDevicePlayback.
(2) localDevice.init will call getPreferredAddress to retrieve the "persist.sys.hdmi.addr.playback" saved in properties. If this device has previously been bound to a Logical address, it will prioritize selecting that address. If not, it will start from the first address of the device (the first address for PLAYBACK devices is 0x4).
(3) localDevices.add will add the initialized localDevice to the list localDevices.

- hdmi_cec_set_option:

During initialization, the HAL layer will be called to enable CEC-related switches, as shown in the table below:

Switch	**Description**
HDMI_OPTION_WAKEUP	When set to false, CEC MSGs like <Image View On> or <Text View On>, which are specified in the CEC protocol to wake up the system, will not wake the system from standby. Since the current version does not implement the CEC wake-up system functionality, this switch has no practical effect.
HDMI_OPTION_ENABLE_CEC	Can be considered the master switch for CEC functionality. When set to false, HAL no longer sends or receives any CEC MSGs, and CEC functionality is disabled. It is controlled through the CEC master switch in the settings menu.
HDMI_OPTION_SYSTEM_CEC_CONTROL	Set to false when the system enters standby and true when waking up. When set to false, it indicates that the Android system no longer processes reported CEC MSGs, handing over to the lower layer for processing. Currently, this switch also has no practical effect, but it may add related functionality when the CEC wake-up system feature is implemented in future versions.

- allocateLogicalAddress:

This step mainly completes the binding of the Logical address.

(1) Call HAL's hdmi_cec_send_message to send POLL MSGs. If a certain number of POLL MSGs are sent without response (NACK), the address is selected for use (the number of retries is determined by HdmiConfig.ADDRESS_ALLOCATION_RETRY).
(2) Call HAL's hdmi_cec_add_logical_address to send the selected Logical address to the lower-level driver.

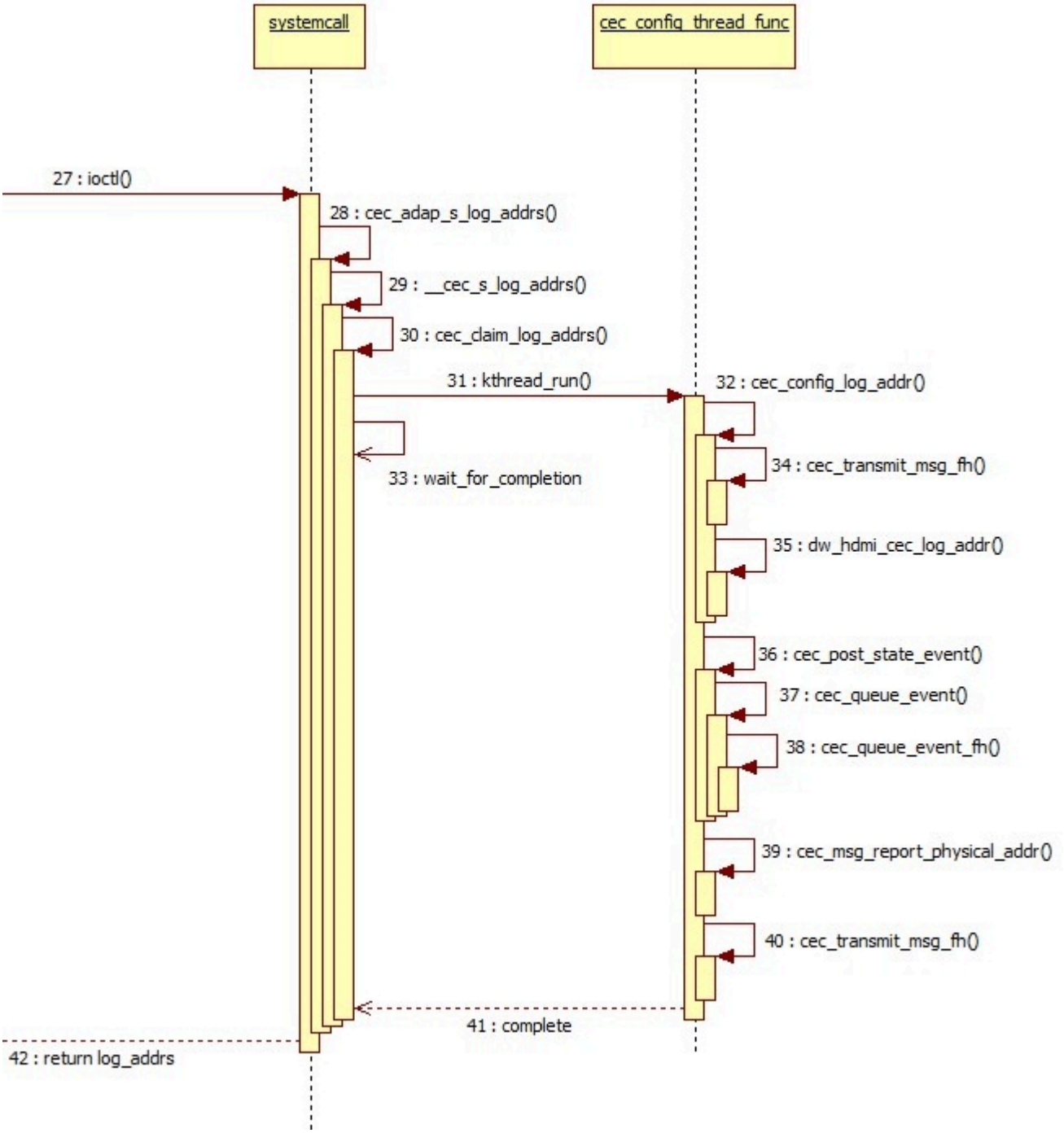
(3) After successful binding, in onAddressAllocated, call the CEC MSG interface to send several CEC MSGs (ReportPhysicalAddress, etc.).

There is an issue in this process: According to the CEC protocol, the binding of Logical addresses requires CEC devices to send POLL MSGs. When a device does not respond to POLL MSGs (NACK), it indicates that the address is not occupied by any device, and the address is selected as the Logical address for this device.

The Android FRAMEWORK, when binding Logical addresses, uses a scheme where HAL's hdmi_cec_send_message is called to directly send several POLL MSGs. If there is no response, the address is selected for binding. Only then is the bound address sent to the driver. However, the kernel CEC driver requires the completion of initialization configuration through IOCTL CEC_ADAP_S_LOG_ADDRS to complete the entire binding process before reporting the bound address. Upper layers are not allowed to directly call the send_message interface to send POLL MSGs before this.

The difference in processes between the two can only be resolved by adapting in the HAL. When the FRAMEWORK calls hdmi_cec_send_message to send POLL MSGs, IOCTL CEC_ADAP_G_LOG_ADDRS is used to obtain the Logical address bound by the kernel driver at this time. If the driver has not yet bound a Logical address (CEC_LOG_ADDR_INVALID), IOCTL CEC_ADAP_S_LOG_ADDRS is called for binding. If the driver has already bound a Logical address and the address is the same as the one sent in the POLL MSG by the upper layer, NACK is returned to the upper layer. If the bound address is different, success is returned, and the FRAMEWORK will choose the next address for POLLING until the addresses selected by the upper layer and the kernel are the same.

The initialization setup process of the kernel cec driver is shown in the figure below:



- The HAL layer sets the Logical address by calling IOCTL CEC_ADAP_S_LOG_ADDRS and cec_adap_s_log_addrs.
- In cec_claim_log_addrs, the initialization configuration thread cec_config_thread_func is run and the completion of configuration is awaited (wait_for_completion(&adap->config_completion)).

```
static void cec_claim_log_addrs(struct cec_adapter *adap, bool block)
{
   if (WARN_ON(adap->is_configuring || adap->is_configured))
      return;

   init_completion(&adap->config_completion);

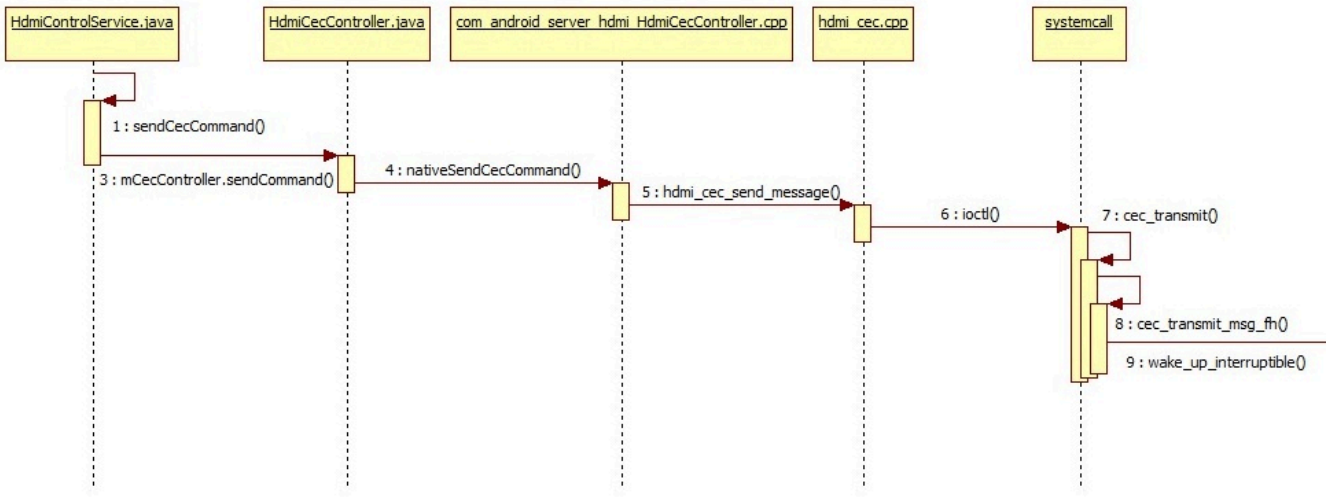
   /* Ready to kick off the thread */
   adap->is_configuring = true;
```

```
    adap->kthread_config = kthread_run(cec_config_thread_func, adap,
                "ceccfg-%s", adap->name);
    if (IS_ERR(adap->kthread_config)) {
        adap->kthread_config = NULL;
    } else if (block) {
        mutex_unlock(&adap->lock);
        wait_for_completion(&adap->config_completion);
        mutex_lock(&adap->lock);
    }
}
```

- In cec_config_thread_func, POLL MSGs are added to the send queue and sent using cec_transmit_msg_fh. If there is no response, the address is bound.
- The bound address is written into the relevant CEC registers by calling adap->ops->adap_log_addr, which calls dw_hdmi_cec_log_addr.
- At this point, the CEC driver initialization setup is complete (adap->is_configured = true), and the event of the CEC driver status change is reported to the upper layer through cec_post_state_event.
- cec_msg_report_physical_addr creates a CEC MSG with OPCODE report physical addr, adds it to the send queue, and sends it.
- Awakens the thread that was waiting for the configuration to complete (complete(&adap->config_completion)).

2.4.2 Introduction to CEC Message Sending Process

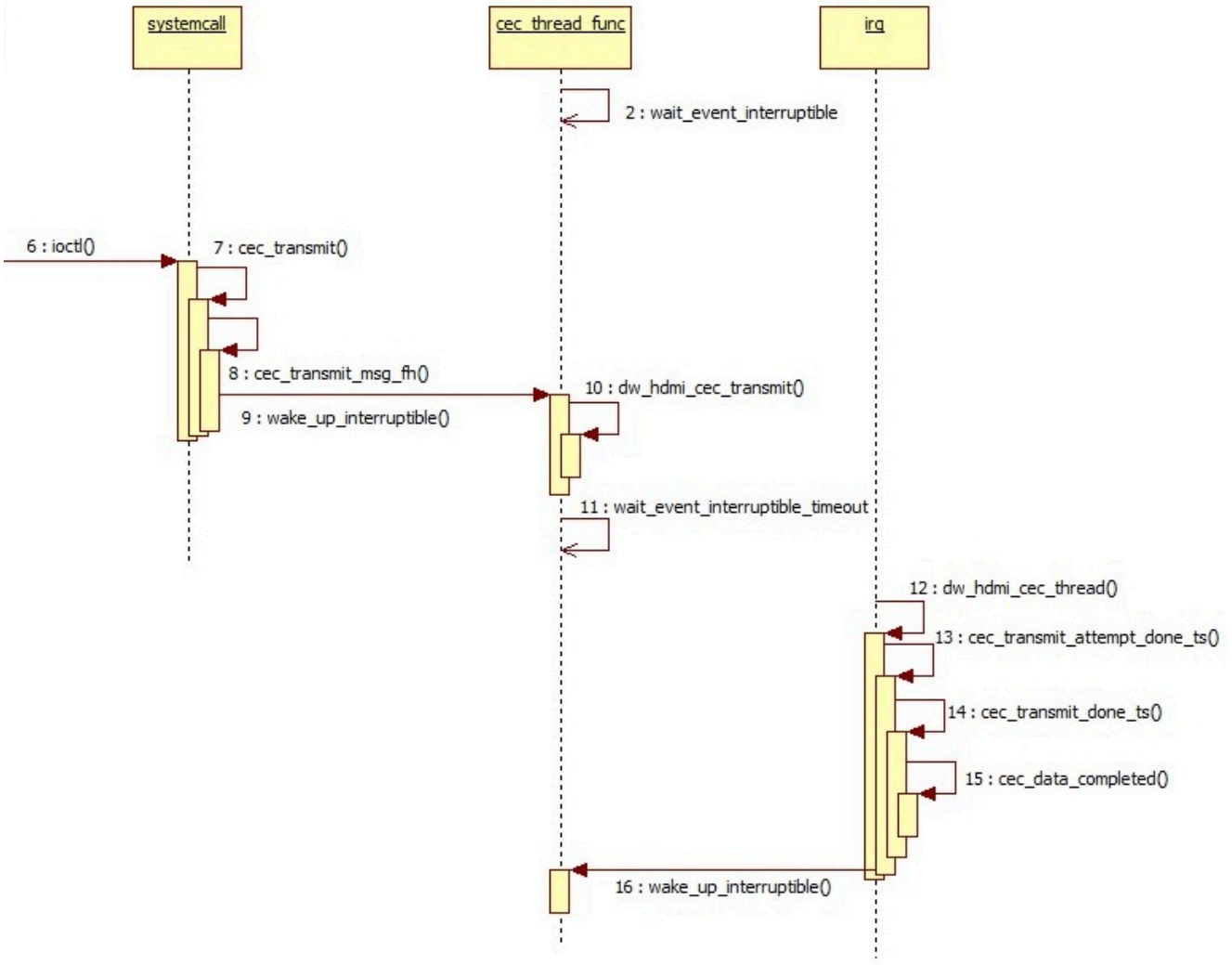
As shown in the figure, the upper layer CEC MSG sending process is relatively simple, just call sendCecCommand to send.



The parameter HdmiCecMessage of sendCecCommand can refer to:

```
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecMessageBuilder.java
```

The kernel process is shown in the figure:



Finally, the HAL layer calls the cec_transmit in the CEC driver through IOCTL CEC_TRANSMIT to send messages.

```
static long cec_transmit(struct cec_adapter *adap, struct cec_fh *fh,
         bool block, struct cec_msg __user *parg)
{
   struct cec_msg msg = {};
   long err = 0;

   if (!(adap->capabilities & CEC_CAP_TRANSMIT))
      return -ENOTTY;
   if (copy_from_user(&msg, parg, sizeof(msg)))
      return -EFAULT;

   /* A CDC-Only device can only send CDC messages */
   if ((adap->log_addrs.flags & CEC_LOG_ADDRS_FL_CDC_ONLY) &&
      (msg.len == 1 || msg.msg[1] != CEC_MSG_CDC_MESSAGE))
      return -EINVAL;

   mutex_lock(&adap->lock);
   if (adap->log_addrs.num_log_addrs == 0)
      err = -EPERM;
   else if (adap->is_configuring)
```

```
        err = -ENONET;
    else if (!adap->is_configured &&
         (adap->needs_hpd || msg.msg[0] != 0xf0))
        err = -ENONET;
    else if (cec_is_busy(adap, fh))
        err = -EBUSY;
    else
        err = cec_transmit_msg_fh(adap, &msg, fh, block);
    mutex_unlock(&adap->lock);
    if (err)
        return err;
    if (copy_to_user(parg, &msg, sizeof(msg)))
        return -EFAULT;
    return 0;
}
```

First, a series of judgment conditions are used to determine whether the CEC driver can send MSG normally at this time. Then, the CEC message MSG to be sent is added to the queue to be sent through cec_transmit_msg_fh. The structure of MSG is as follows:

```
struct cec_msg {
        __u64 tx_ts;
        __u64 rx_ts;
        __u32 len;
        __u32 timeout;
        __u32 sequence;
        __u32 flags;
        __u8 msg[CEC_MAX_MSG_SIZE];
        __u8 reply;
        __u8 rx_status;
        __u8 tx_status;
        __u8 tx_arb_lost_cnt;
        __u8 tx_nack_cnt;
        __u8 tx_low_drive_cnt;
        __u8 tx_error_cnt;
};
```

The explanation of the structure members is as follows:

Structure Member	Description
tx_ts	Timestamp in nanoseconds, which is set after the CEC driver completes sending MSG.
rx_ts	Timestamp in nanoseconds, which is set after the CEC driver completes receiving MSG.
len	The length of MSG.
sequence	The CEC driver framework assigns a sequence number to the message being sent. This can be used to track the reply to previously sent messages.
msg	The actual payload of the CEC MSG.
reply	Used only when sending CEC MSG. If it is not 0, the CEC driver will wait for a reply to this message after sending it (for example, after sending , according to the CEC protocol, a reply will be received). Currently, it is set to 0 and will not wait specifically in the CEC driver for the MSG response.
rx_status	The CEC driver will set it after completing the reception of MSG, marking the reception status: CEC_RX_STATUS_OK CEC_RX_STATUS_TIMEOUT CEC_RX_STATUS_FEATURE_ABORT
tx_status	The CEC driver will set it after completing the sending of MSG, marking the sending status: CEC_TX_STATUS_OK CEC_TX_STATUS_ARB_LOST CEC_TX_STATUS_NACK CEC_TX_STATUS_LOW_DRIVE CEC_TX_STATUS_ERROR CEC_TX_STATUS_MAX_RETRIES
tx_arb_lost_cnt	After the CEC MSG is sent, the number of Arbitration Lost errors is counted.
tx_nack_cnt	After the CEC MSG is sent, the number of Not Acknowledged errors is counted.
tx_low_drive_cnt	After the CEC MSG is sent, the number of Low Drive Detected errors is counted.
tx_error_cnt	After the CEC MSG is sent, the number of Error errors is counted.

- cec_thread_func is the thread that manages the CEC MSG sending queue. When there is no MSG being sent at this time, the thread waits for a new MSG to join the queue.

```
wait_event_interruptible(adap->kthread_waitq,
    kthread_should_stop ||
    (!adap->transmitting &&
     !list_empty(&adap->transmit_queue)));
```

- After cec_transmit_msg_fh adds the message to be sent to the queue, if there is no MSG being sent at this time, it will wake up cec_thread_func to send it. If there is a MSG being sent at this time, it will add MSG to the waiting queue and wait for the sending to be completed.

```
if (fh)
    list_add_tail(&data->xfer_list, &fh->xfer_list);

list_add_tail(&data->list, &adap->transmit_queue);
adap->transmit_queue_sz++;
if (!adap->transmitting)
    wake_up_interruptible(&adap->kthread_waitq);

/* All done if we don't need to block waiting for completion */
if (!block)
    return 0;

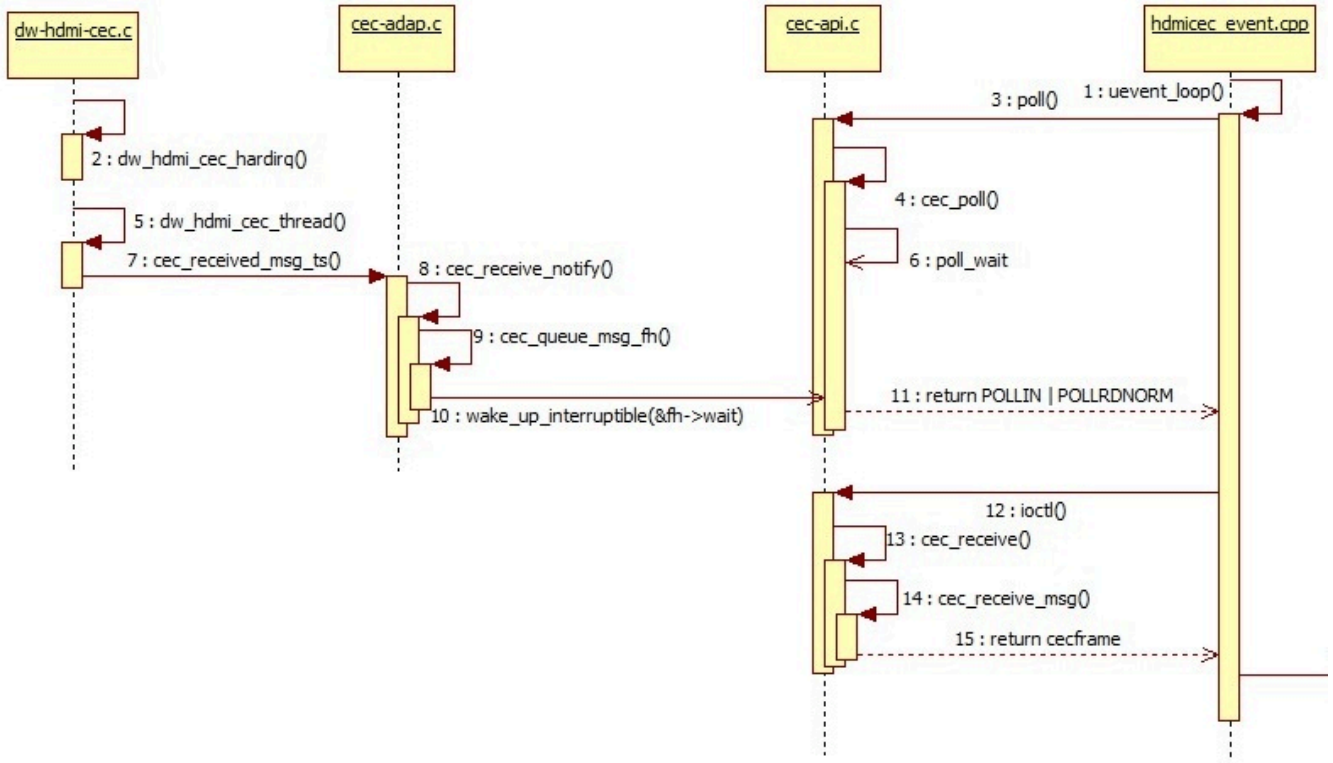
/*
 * If we don't get a completion before this time something is really
 * wrong and we time out.
 */
timeout = CEC_XFER_TIMEOUT_MS;
/* Add the requested timeout if we have to wait for a reply as well */
if (msg->timeout)
    timeout += msg->timeout;

/*
 * Release the lock and wait, retake the lock afterwards.
 */
mutex_unlock(&adap->lock);
res = wait_for_completion_killable_timeout(&data->c,
                    msecs_to_jiffies(timeout));
mutex_lock(&adap->lock);

if (data->completed) {
    /* The transmit completed (possibly with an error) */
    *msg = data->msg;
    kfree(data);
    return 0;
}
```

- After that, through the callback adap->ops->adap_transmit, dw_hdmi_cec_transmit is called to configure the CEC register to start sending CEC MSG.
- After the sending is completed, the CEC interrupt is generated, and dw_hdmi_cec_hardirq reads the register, records the status of this sending such as sending success or some error and records it in tx_status.
- In cec_transmit_done_ts, the errors of this sending are counted. If there is an error, it will be resent according to the retry attempts. If there is no error in this sending, cec_data_completed is called to complete this sending and wake up cec_thread_func to send the next CEC MSG.

2.4.3 Introduction to CEC Message Reception Process



- When a CEC MSG is received, a CEC interrupt is generated, and dw_hdmi_cec_hardirq reads the CEC RX DATA register to obtain the received CEC MSG.
- cec_received_msg_ts will determine whether the received CEC MSG is usable.
- In cec_receive_notify, when the CEC driver is not set to passthrough mode, it will automatically reply to the received CEC MSG according to the CEC protocol. Since our solution always operates in passthrough mode, it will call cec_queue_msg_fh to store the received CEC MSG in the queue and wake up poll_wait, notifying the upper layer that a CEC MSG has been received.

```
static void cec_queue_msg_fh(struct cec_fh *fh, const struct cec_msg *msg)
{
    static const struct cec_event ev_lost_msgs = {
        .event = CEC_EVENT_LOST_MSGS,
        .lost_msgs.lost_msgs = 1,
    };
    struct cec_msg_entry *entry;

    mutex_lock(&fh->lock);
    entry = kmalloc(sizeof(*entry), GFP_KERNEL);
    if (entry) {
        entry->msg = *msg;
        /* Add new msg at the end of the queue */
        list_add_tail(&entry->list, &fh->msgs);

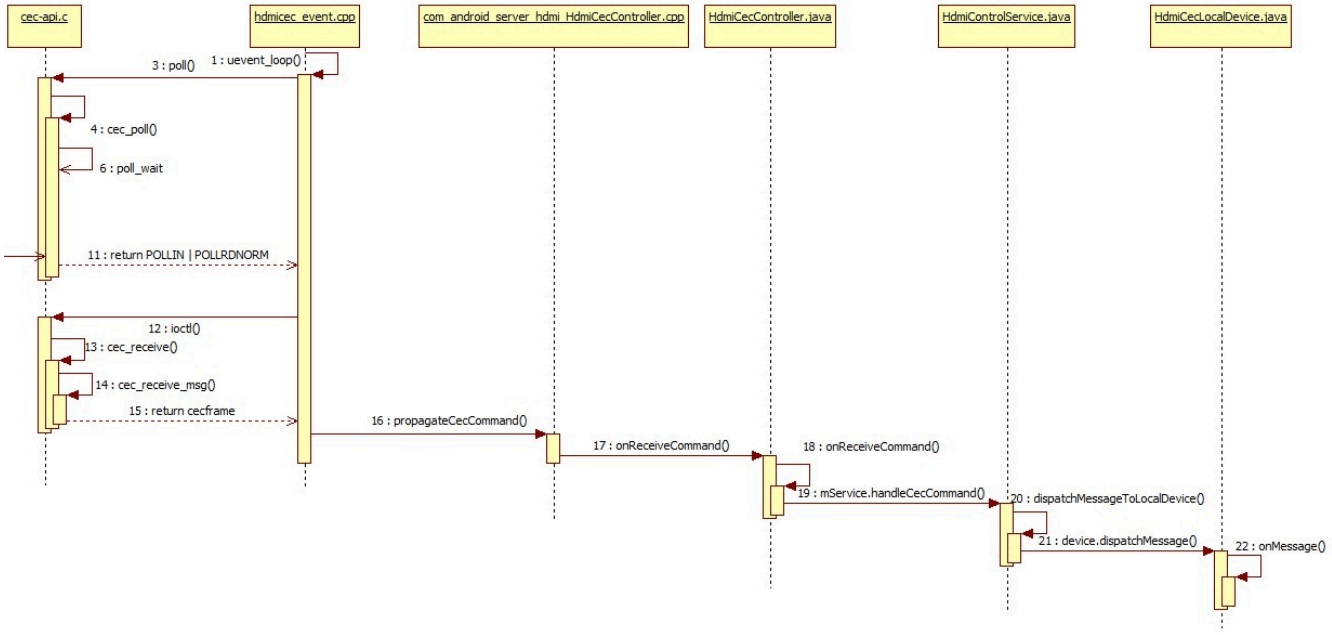
        if (fh->queued_msgs < CEC_MAX_MSG_RX_QUEUE_SZ) {
            /* All is fine if there is enough room */
            fh->queued_msgs++;
            mutex_unlock(&fh->lock);
```

```
            wake_up_interruptible(&fh->wait);
            return;
        }

        /*
         * if the message queue is full, then drop the oldest one and
         * send a lost message event.
         */
        entry = list_first_entry(&fh->msgs, struct cec_msg_entry, list);
        list_del(&entry->list);
        kfree(entry);
    }
    mutex_unlock(&fh->lock);

    /*
     * We lost a message, either because kmalloc failed or the queue
     * was full.
     */
    cec_queue_event_fh(fh, &ev_lost_msgs, ktime_get_ns);
}
```

The HAL layer's event listening thread uevent_loop, upon receiving the event of receiving a CEC MSG, obtains the CEC MSG stored in the MSG queue through IOCTL CEC_RECEIVE.



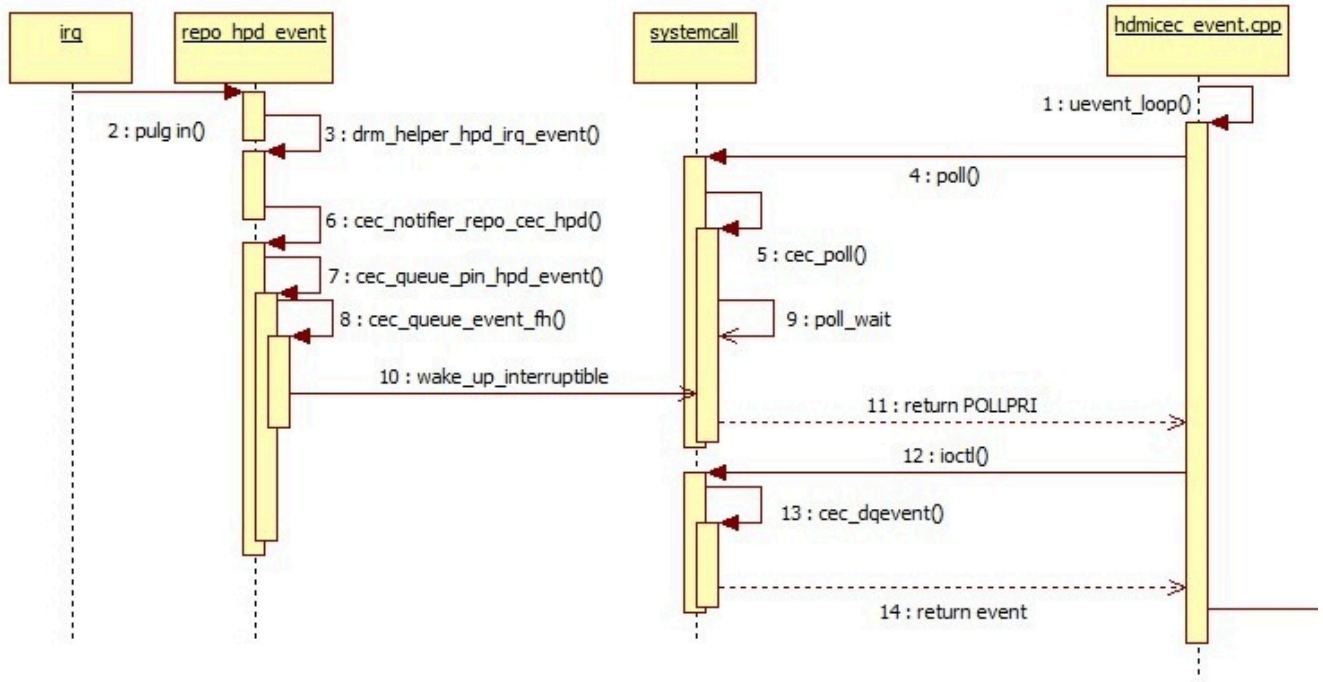
As shown in the figure, the FRAMEWORK's handling of received CEC messages is also relatively simple. It reports the received CEC MSG layer by layer, and finally, in onMessage, it distributes the CEC MSG to the corresponding handle function for processing based on the OPCODE of the CEC MSG.

2.4.4 CEC Event Handling Process

The CEC driver reports events to notify the upper layer of the current state changes. The types of events currently implemented are shown in the table below:

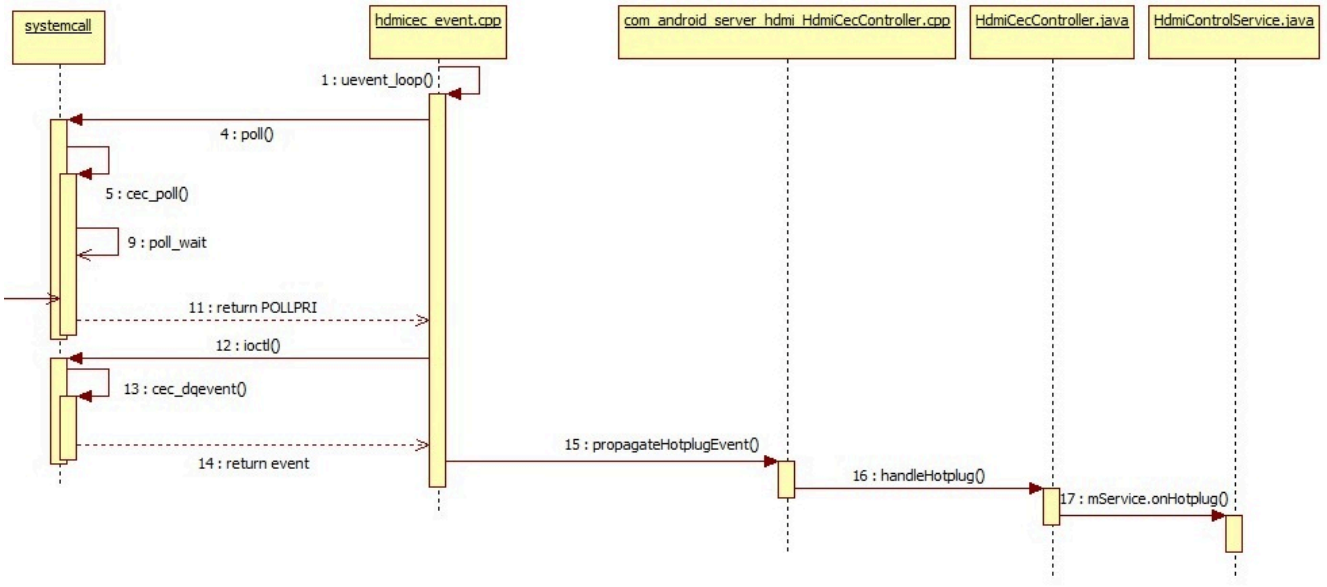
Name	Description
CEC_EVENT_STATE_CHANGE	The state of the adapter has changed, such as from the configured state to the unconfigured state.
CEC_EVENT_LOST_MSGS	CEC receives MSGs in the MSG queue that are not read in time, causing MSGs to be lost.
CEC_EVENT_PIN_CEC_LOW	The CEC pin becomes low, and this event is not currently used.
CEC_EVENT_PIN_CEC_HIGH	The CEC pin becomes high, and this event is not currently used.
CEC_EVENT_PIN_HPD_LOW	The HPD pin of HDMI becomes low, usually indicating that HDMI is unplugged.
CEC_EVENT_PIN_HPD_HIGH	The HPD pin of HDMI becomes high, usually indicating that HDMI is inserted.

Taking the HDMI hot-plug event as an example, as shown in the figure:



- When HDMI is plugged or unplugged, an HDMI interrupt is generated, calling cec_queue_pin_hpd_event to create a new HPD CEC EVENT.
- cec_queue_event_fh adds this event to the CEC EVENT list and wakes up poll_wait, reporting the event.
- The HAL layer calls cec_dqevent through IOCTL CEC_DQEVENT to retrieve the current event from the

EVENT list. The event reporting process is completed.



- After the HAL layer obtains the event, it will handle different events, and the HPD event will continue to be reported.
- Eventually, it will be processed in onHotplug. If this is a plug-in event, allocateLogicalAddress will be called to rebind the Logical address, and the process is essentially the same as in 2.3.1.2.

2.5 New CEC FEATURE Common Interface

Android native code already supports multiple FEATURES of the CEC protocol. If you want to add a FEATURE that is not yet supported, you can use the following interfaces:

- Proactively send CEC MSG:

frameworks/base/services/core/java/com/android/server/hdmi/HdmiControlService.java

```
/**
 * Transmit a CEC command to CEC bus.
 *
 * @param command CEC command to send out
 * @param callback interface used to the result of send command
 */
@ServiceThreadOnly
void sendCecCommand(HdmiCecMessage command, @Nullable SendMessageCallback callback) {
    assertRunOnServiceThread;
    if (mMessageValidator.isValid(command) == HdmiCecMessageValidator.OK) {
        mCecController.sendCommand(command, callback);
    } else {
        HdmiLogger.error("Invalid message type:" + command);
        if (callback != null) {
            callback.onSendCompleted(SendMessageResult.FAIL);
        }
    }
```

```
    }

    @ServiceThreadOnly
    void sendCecCommand(HdmiCecMessage command) {
        assertRunOnServiceThread;
        sendCecCommand(command, null);
    }
```

The parameter HdmiCecMessage command can be directly created by calling the corresponding interface in the following path:

```
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecMessageBuilder.java
```

```
    @Override
    @ServiceThreadOnly
    protected void onStandby(boolean initiatedByCec, int standbyAction) {
        assertRunOnServiceThread;
        if (!mService.isControlEnabled || initiatedByCec || !mAutoTvOff) {
            return;
        }
        switch (standbyAction) {
            case HdmiControlService.STANDBY_SCREEN_OFF:
                Slog.e(TAG, "send standby");
                /* cec cts specification requires that standby message must be broadcast */
                mService.sendCecCommand(
                        HdmiCecMessageBuilder.buildStandby(mAddress, Constants.ADDR_BROADCAST));
                break;
            case HdmiControlService.STANDBY_SHUTDOWN:
                // ACTION_SHUTDOWN is taken as a signal to power off all the devices.
                mService.sendCecCommand(
                        HdmiCecMessageBuilder.buildStandby(mAddress, Constants.ADDR_BROADCAST));
                break;
        }
    }
```

In the above code, to implement the function of the box to put the TV into standby, according to the CEC protocol, a standby CEC MSG needs to be sent. Therefore, the sendCecCommand interface is called to send it, and the parameter standby's CEC MSG is directly created by calling the buildStandby interface of HdmiCecMessageBuilder.

- Add processing for CEC MSG:

Since the system will automatically receive CEC MSG and report it, when the received CEC MSG is not supported by the system for corresponding processing, a new processing method needs to be added.
As shown in the following code, after receiving the CEC MSG, it will be processed in onMessage, where handlexxxxx is the processing for the corresponding MSG, and message.getOpcode is to get the OPCODE of the CEC MSG. Choose the corresponding processing based on the OPCODE.

```
frameworks/base/services/core/java/com/android/server/hdmi/HdmiCecLocalDevice.java
```

```
```

```
@ServiceThreadOnly
protected final boolean onMessage(HdmiCecMessage message) {
    assertRunOnServiceThread;
    if (dispatchMessageToAction(message)) {
        return true;
    }
    switch (message.getOpcode) {
        case Constants.MESSAGE_ACTIVE_SOURCE:
            return handleActiveSource(message);
        case Constants.MESSAGE_INACTIVE_SOURCE:
            return handleInactiveSource(message);
        case Constants.MESSAGE_REQUEST_ACTIVE_SOURCE:
            return handleRequestActiveSource(message);
        case Constants.MESSAGE_GET_MENU_LANGUAGE:
            return handleGetMenuLanguage(message);
        case Constants.MESSAGE_SET_MENU_LANGUAGE:
            return handleSetMenuLanguage(message);
        case Constants.MESSAGE_GIVE_PHYSICAL_ADDRESS:
            return handleGivePhysicalAddress();
        case Constants.MESSAGE_GIVE_OSD_NAME:
            return handleGiveOsdName(message);
        case Constants.MESSAGE_GIVE_DEVICE_VENDOR_ID:
            return handleGiveDeviceVendorId();
        case Constants.MESSAGE_GET_CEC_VERSION:
            return handleGetCecVersion(message);
        case Constants.MESSAGE_REPORT_PHYSICAL_ADDRESS:
            return handleReportPhysicalAddress(message);
        case Constants.MESSAGE_ROUTING_CHANGE:
            return handleRoutingChange(message);
        case Constants.MESSAGE_ROUTING_INFORMATION:
            return handleRoutingInformation(message);
        case Constants.MESSAGE_INITIATE_ARC:
            return handleInitiateArc(message);
        case Constants.MESSAGE_TERMINATE_ARC:
            return handleTerminateArc(message);
        case Constants.MESSAGE_SET_SYSTEM_AUDIO_MODE:
            return handleSetSystemAudioMode(message);
        case Constants.MESSAGE_SYSTEM_AUDIO_MODE_STATUS:
            return handleSystemAudioModeStatus(message);
        case Constants.MESSAGE_REPORT_AUDIO_STATUS:
            return handleReportAudioStatus(message);
        case Constants.MESSAGE_STANDBY:
            return handleStandby(message);
        case Constants.MESSAGE_TEXT_VIEW_ON:
            return handleTextViewOn(message);
        case Constants.MESSAGE_IMAGE_VIEW_ON:
            return handleImageViewOn(message);
        case Constants.MESSAGE_USER_CONTROL_PRESSED:
            return handleUserControlPressed(message);
        case Constants.MESSAGE_USER_CONTROL_RELEASED:
            return handleUserControlReleased();
        case Constants.MESSAGE_SET_STREAM_PATH:
            return handleSetStreamPath(message);
```

```
            case Constants.MESSAGE_GIVE_DEVICE_POWER_STATUS:
                return handleGiveDevicePowerStatus(message);
            case Constants.MESSAGE_MENU_REQUEST:
                return handleMenuRequest(message);
            case Constants.MESSAGE_MENU_STATUS:
                return handleMenuStatus(message);
            case Constants.MESSAGE_VENDOR_COMMAND:
                return handleVendorCommand(message);
            case Constants.MESSAGE_VENDOR_COMMAND_WITH_ID:
                return handleVendorCommandWithId(message);
            case Constants.MESSAGE_SET_OSD_NAME:
                return handleSetOsdName(message);
            case Constants.MESSAGE_RECORD_TV_SCREEN:
                return handleRecordTvScreen(message);
            case Constants.MESSAGE_TIMER_CLEARED_STATUS:
                return handleTimerClearedStatus(message);
            case Constants.MESSAGE_REPORT_POWER_STATUS:
                return handleReportPowerStatus(message);
            case Constants.MESSAGE_TIMER_STATUS:
                return handleTimerStatus(message);
            case Constants.MESSAGE_RECORD_STATUS:
                return handleRecordStatus(message);
            default:
                return false;
        }
    }
```

Therefore, to support a new CEC MSG, simply add the corresponding handle in the switch statement.

3. Common Debug Methods

When the CEC function does not operate normally, developers can follow these steps for DEBUG:

- Execute the DEBUG script to capture the status of the DEBUG node and LOG.
- Check the current status of the DEBUG node to confirm whether CEC is correctly initialized and operating normally.
- Verify whether CEC MSG is successfully sent and received by checking the kernel log.
- Confirm that each CEC FEATURE is correctly processed by checking the Android log.

3.1 DEBUG Script

Below is the DEBUG script for CEC, with the script code as follows:

```
#!/system/bin/sh

log ************start CEC debug script**************
```

```
echo 2 > /sys/module/cec/parameters/debug

if [ -f "/data/cec_status.txt" ]
then
rm /data/cec_status.txt
fi

if [ -f "/data/cec_log.txt" ]
then
rm /data/cec_log.txt
fi

cat /sys/kernel/debug/cec/cec0/status >> /data/cec_status.txt
cat /sys/kernel/debug/dw-hdmi/ctrl >> /data/cec_status.txt

logcat -b all -f /data/cec_log.txt
```

Usage:

- Push the `cec_debug.sh` to the `/system/bin` directory.
- Execute `/system/bin/cec_debug.sh` before reproducing the abnormal condition.
- The log "********start CEC debug script*********" marks the start of the script's execution.
- Perform the operations that lead to the anomaly.
- The status of the relevant CEC debug nodes will be generated in `/data/cec_status.txt`.
- The synchronized kernel log and logcat will be generated in `/data/cec_log.txt`.

3.1.1 Log Content Introduction

The script captures content includes the status of various DEBUG nodes and log sections. For detailed explanation of the node content, see section 3.1.2 CEC DEBUG Node Description.
The log primarily needs to focus on three parts:

Logs containing the keyword `cec-dw_hdmi` are kernel logs:

```
04-11 11:44:50.969     0     0 I cec-dw_hdmi: cec_transmit_msg_fh: 44
```

These can also be manually captured by entering the following command:

```
dmesg | grep cec-dw_hdmi
```

Logs containing the keyword `hdmicec` are logs from the HAL layer:

```
04-11 12:39:59.015   281   281 I hdmicec : hdmi_cec_set_option: system_control: value: 0
```

These can also be manually captured by entering the following command:

```
logcat | grep hdmicec
```

Logs containing the keyword `Hdmi` are logs from the framework layer:

```
04-11 11:44:50.565   391   494 W HdmiCecController: Failed to send <Give Device Power Status> src: 4, dst: 0 with errorCode=-1
```

These can also be manually captured by entering the following command:

```
logcat | grep Hdmi
```

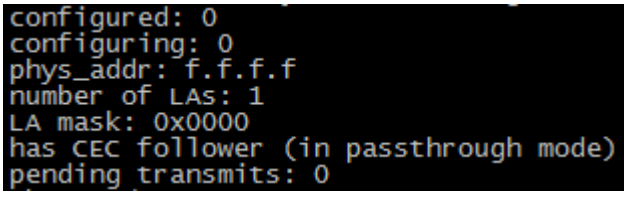
For detailed common analysis methods of logs, see section 3.2 Troubleshooting.

3.1.2 CEC DEBUG Node Description

The DEBUG node path for CEC operational status is:

```
/sys/kernel/debug/cec/cec0/status
```

After executing `cat`, the display is:



The explanation of each parameter is as follows in the table:

Name	Description
configured	1: CEC has been correctly configured. 0: CEC has not been correctly configured.
configuring	1: CEC is being configured 0: CEC has not started to be configured or has been configured.
phys_addr	Physical address, read from the TV EDID. If it is f.f.f.f, it indicates that CEC is uninitialized or failed to obtain the physical address.
LA mask	Logical address mask. When CEC is not initialized, it is 0x0000. Under normal circumstances, right-shift the value until bit 0 becomes 1; the number of shifted bits equals the logical address. For example: 0x0010 >> 4 = 0x1, so the logical address is 4 (i.e., 0x4).
has CEC follower (in passthrough mode)	Indicates that the CEC driver does not handle directly.
pending transmits	The number of CEC MSGs waiting to be sent.

HDMI Controller Register Viewing Node:

```
/sys/kernel/debug/dw-hdmi/ctrl
```

`cat` this node can view the HDMI controller registers, and you can pay attention to the CEC-related registers. For details on the related registers, please refer to the datasheet of the HDMI controller.

3.1.3 How to Enable CEC Kernel Log

```
echo 2 > /sys/module/cec/parameters/debug
```

Enables logging for CEC MSG transmission and reception.

```
echo 1 > /sys/module/cec/parameters/debug
```

Enables other CEC logs.

3.2 Troubleshooting

3.2.1 Scenarios Where All CEC Functions Are Inoperative

The following items also belong to the confirmation items when some functions cannot be used. When some CEC functions are inoperative, the following items should also be confirmed.

- Confirm whether the CEC switch in Settings > Input is turned on.
- Confirm whether there is "hdmi_cec_device_open open error!" in the log, and execute ls -l /dev/cec0.

```
130|rk3328_box:/ # ls -l /dev/cec0

crw-rw---- 1 system system 250,  0 2016-01-21 08:50 /dev/cec0
```

Confirm whether the permissions of cec0 are as shown above or higher than the above.
If the permissions are too low, please confirm whether the following modifications are made:

```
diff --git a/ueventd.rockchip.rc b/ueventd.rockchip.rc
index a7d1356..9eb824d 100755
--- a/ueventd.rockchip.rc
+++ b/ueventd.rockchip.rc
@@ -96,6 +96,7 @@

 #for hdmi cec
 /dev/cec   0666   system  system
+/dev/cec0  0660   system  system
 #sofia
 /dev/dcc   0666   system  system
 /dev/pmem_gfx   0666   system  system
```

- Confirm that the status in /sys/kernel/debug/cec/cec0/status is normal (see 3.1.2). Commonly, it may be caused by the following abnormalities:

(1) phys_addr is f.f.f.f: Please confirm whether the EDID of HDMI is read normally, or whether the TV supports CEC.
(2) LA mask is 0x0000: Binding Logical address failure.
In this case, first confirm whether there is the following log:

```
04-11 08:56:51.764   296   296 E hdmicec : can't make kernel addr done
```

If there is this log, it is necessary to confirm whether EDID is read correctly.
If there is no log above, it is necessary to confirm the following log:

```
04-11 08:56:51.764   296   296 D hdmicec : kernel logic addr:ff, preferred logic addr:04
04-11 08:56:51.764   296   296 I hdmicec : kernel logaddr is not existing
04-11 08:56:51.764   296   296 D hdmicec : set_kernel_logical_address, logic addr:04
```

Among them, logic addr is not necessarily 04, but it must be one of 04, 08, 0b. If there is no log above, it is necessary to trace the following file for further analysis:

```
hardware/rockchip/hdmicec/hdmi_cec.cpp
```

On the basis of the existence of the above log, confirm whether there is the following log:

```
04-11 08:35:37.297     0     0 I cec-dw_hdmi: new physical address 3.0.0.0
04-11 08:35:37.297     0     0 I cec-dw_hdmi: physical address: 3.0.0.0, claim 1 logical addresses
```

If there is none, it is necessary to trace the cec_adap_s_log_addrs process in the following path for analysis:

```
kernel/drivers/media/cec/cec-api.c
```

Then confirm whether there is the following log. The address in the first log is the address of a POLL MSG. It is not necessarily 44, it may also be 88, bb, and there may be three addresses of logs that appear. It is necessary to confirm how many CEC devices are connected to the TV, and whether the quantity matches the quantity of addresses. For detailed procedures, please refer to the CEC section of the HDMI protocol.
It is important to focus on the status in the second log. If it is confirmed that the address is not occupied by other CEC devices, and the status is not 04 (nack), it is necessary to combine the HDMI ctrl register and CEC waveform for analysis.

```
04-11 08:35:37.297     0     0 I cec-dw_hdmi: cec_transmit_msg_fh: 44
04-11 08:35:37.326     0     0 I cec-dw_hdmi: cec_transmit_done_ts: status 04
```

3.2.2 TV in Standby Mode, Box Not Entering Standby

- Due to Android's default behavior of disabling the box from entering standby mode through CEC, developers can set the property `persist.sys.hdmi.keep_awake` to false. Alternatively, make the following modification to set the default value of this property to false:

```
diff --git a/services/core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java
b/services/core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java
--- a/services/core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java
+++ b/services/core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java
@@ -210,7 +210,7 @@
    private ActiveWakeLock getWakeLock {
        assertRunOnServiceThread;
        if (mWakeLock == null) {
-           if (SystemProperties.getBoolean(Constants.PROPERTY_KEEP_AWAKE, true)) {
+           if (SystemProperties.getBoolean(Constants.PROPERTY_KEEP_AWAKE, false)) {
                mWakeLock = new SystemWakeLock;
            } else {
                // Create a dummy lock object that doesn't do anything about wake lock,
```

- Check the captured log to see if the following is present, to verify if the box has received the standby MSG (0x36):

```
[ 3465.670484] cec-dw_hdmi: cec_received_msg_ts: 0f 36
[ 3465.670617] cec-dw_hdmi: cec_receive_notify: 0f 36
01-21 09:47:47.675    0    0 I cec-dw_hdmi: cec_received_msg_ts: 0f 36
04-11 09:16:32.963  299  342 D hdmicec : poll revent:41
01-21 09:47:47.675    0    0 I cec-dw_hdmi: cec_receive_notify: 0f 36
04-11 09:16:32.963  299  342 D hdmicec : poll receive msg
04-11 09:16:32.963  299  342 D hdmicec : poll receive msg[0]:0f
04-11 09:16:32.963  299  342 D hdmicec : poll receive msg[1]:36
```

If it is not present, then an oscilloscope should be used to examine the CEC bus waveform to determine if the TV indeed sent the standby MSG.

3.2.3 Box in Standby, TV Not Entering Standby

- If the box is put into standby mode immediately after the TV is turned on, and the TV does not enter standby, please wait for 15 seconds after the TV is powered on before attempting to put the box into standby mode. This is because some TVs do not respond to the standby MSG sent by the box for a period of time immediately after powering on.
- Check for the following logs to determine if the CEC standby process is normal:

```
04-11 08:37:13.823  394  394 V HdmiControlService: On standby-action cleared:4
04-11 08:37:13.823  394  394 V HdmiControlService: onStandbyCompleted
04-11 08:37:13.927  296  296 I hdmicec : send msg LEN:2,opcode:36,addr:4f
```

If the above logs are missing or only the first two are present, you need to trace the code in the following file path for further analysis:

```
frameworks/base/services/core/java/com/android/server/hdmi/HdmiControlService.java
```

- On the basis of the previous step, confirm the following logs to judge whether the standby message was sent successfully:

```
04-11 08:37:13.928    0    0 I cec-dw_hdmi: cec_transmit_msg_fh: 4f 36
04-11 08:37:13.991    0    0 I cec-dw_hdmi: cec_transmit_done_ts: status 01
```

If the above logs are missing, it is necessary to confirm whether hdmi_cec_send_message is correctly called in the following path, and whether it successfully calls IOCTL CEC_TRANSMIT:

```
hardware/rockchip/hdmicec/hdmi_cec.cpp
```

If both of the above logs exist, but the value of cec_transmit_done_ts: status is not 01, it indicates that an exception occurred on the CEC bus during transmission, and further analysis is required in conjunction with HDMI control registers and CEC signals.

3.2.4 Box Wakes Up, TV Not Woken Up

- Confirm whether the box is properly awakened, refer to section 3.2.1 to check if CEC initialization is without any anomalies.
- Check for the following logs to determine if the CEC wake-up message has been sent and transmitted successfully:

```
04-11 09:27:37.540    0    0 I cec-dw_hdmi: cec_transmit_msg_fh: 40 0d
04-11 09:27:37.593    0    0 I cec-dw_hdmi: cec_transmit_done_ts: status 01
04-11 09:27:37.594    0    0 I cec-dw_hdmi: cec_transmit_msg_fh: 4f 82 30 00
04-11 09:27:37.707    0    0 I cec-dw_hdmi: cec_transmit_done_ts: status 01
```

If these two logs are present but the transmission fails (status is not 01), further analysis is required in conjunction with the HDMI control registers and CEC signals.
If these two logs are not present, it is necessary to trace the sending process of ActiveSource and TextViewOn, these two CEC MSGs, starting from onAddressAllocated in the following path to check for any errors.

```
core/java/com/android/server/hdmi/HdmiCecLocalDevicePlayback.java
```

3.2.5 TV Remote Control Cannot Control the Box's UI

This issue, due to the significant differences between different brands and models of TVs, does not have a universal solution. Here, only some experiences are provided, which may not be applicable to all TVs.

- Observe whether the TV has a dedicated CEC input source channel, such as some Samsung TVs that need to switch to the Anynet input channel to use this feature.

- After the TV is powered on and reaches a stable state, send an ActiveSource MSG to the TV, which can call the system's one-touch-play interface.

```
frameworks/base/services/core/java/com/android/server/hdmi/OneTouchPlayAction.java
```

4. Reference Documents

"HDMI Specification 1.4b.pdf"

"Rockchip Android 5.x HDMI CEC Software Description.doc"

HDMI-CEC Control Service